

INSTITUT SUPÉRIEUR D'INFORMATIQUE

Master 1 — Développement de Systèmes d'Information Avancés (DSIA)

Cours pratique de Supervised Learning

[Emplacement suggéré pour le logo ISI]

PROJET FINAL — CAPSTONE

« De la Donnée à la Décision »

D'un jeu de données brut à un modèle prédictif en production

Prédiction du churn client — Telco Customer Churn Dataset

Projet réalisé par

MOURTALLA GUEYE

Encadrant : Ibrahima SY — Enseignant chercheur

Année universitaire 2025 – 2026

Sommaire

Introduction.....	3
Mission 0 — Cadrage du problème.....	3
Mission 1 — Données et analyse exploratoire.....	3
Mission 2 — Pipeline sans fuite de données.....	6
Mission 3 — Modélisation et comparaison.....	6
Mission 4 — Optimisation et interprétabilité.....	8
Mission 5 — Déploiement, tests et monitoring.....	10
Questions de réflexion.....	11
Conclusion.....	12
Bibliographie et ressources.....	12
Annexe A — Dépôt Git et reproduction.....	12

Introduction

Ce rapport présente la réalisation complète du projet final du cours pratique de Supervised Learning : la construction d'un système de machine learning de bout en bout, depuis la donnée brute jusqu'à un modèle prédictif déployé en production, appliqué à un problème métier concret de rétention client.

Un opérateur de télécommunications souhaite anticiper les résiliations (churn) de ses clients afin de cibler, en priorité, une campagne de rétention. L'objectif n'est pas de maximiser un score de performance de façon isolée, mais de démontrer une démarche professionnelle complète : cadrer le problème métier avant d'écrire la moindre ligne de code, explorer et comprendre les données, construire un pipeline strictement exempt de fuite de données, comparer honnêtement plusieurs familles de modèles, optimiser et interpréter le modèle retenu, puis le déployer sous forme d'API testée et documentée. Le fil conducteur imposé par le sujet — respecter l'enchaînement méthodologique d'un projet ML réel, en particulier séparer les données d'entraînement et de test avant toute transformation apprise — constitue le critère d'évaluation central de ce travail.

Mission 0 — Cadrage du problème

Avant toute ligne de code, cette mission fixe le cadre métier, les coûts d'erreur, la métrique d'évaluation et le seuil de réussite du projet — décisions prises une fois pour toutes, en amont de la modélisation, pour éviter que le choix des métriques ne soit orienté a posteriori par les résultats obtenus.

Problème métier

L'opérateur télécom souhaite identifier, avant leur résiliation, les clients à risque de churn afin que le service marketing puisse leur adresser en priorité une offre de rétention personnalisée. La variable Churn est définie comme : le client a résilié son contrat au cours de la période d'observation (Churn = Yes) ou non (Churn = No).

Coûts d'erreur

Un faux négatif (client qui part sans avoir été ciblé) entraîne la perte de la valeur vie client restante, estimée entre 800 et 1000 € (valeur future perdue et coût d'acquisition d'un nouveau client, généralement cinq fois plus élevé que la rétention). Un faux positif (offre envoyée à un client fidèle) coûte le prix de la remise commerciale, estimé entre 30 et 50 €. Le faux négatif coûte donc environ 15 à 20 fois plus cher que le faux positif — ce déséquilibre justifie de privilégier le rappel de la classe churn, quitte à accepter davantage de faux positifs.

Métrique et seuil de réussite

- Métrique principale : F2-score sur la classe churn (pondère le rappel deux fois plus que la précision), cohérent avec l'asymétrie de coûts.
- Métriques secondaires : ROC-AUC et précision de la classe churn.
- Seuil de réussite fixé a priori : F2-score $\geq 0,65$ et rappel $\geq 0,75$ sur le jeu de test, en battant significativement la baseline naïve.

Risques et hypothèses

- Stationnarité : les comportements de churn observés sont supposés stables à court terme ; un changement tarifaire ou concurrentiel peut invalider cette hypothèse.
- Latence : usage principal en batch mensuel, sans contrainte temps réel forte.
- RGPD et explicabilité : données pseudonymisées, aucune donnée directement identifiante utilisée comme feature ; explicabilité requise pour l'équipe marketing.

Mission 1 — Données et analyse exploratoire

Cette mission établit une compréhension fine du dataset Telco Customer Churn (IBM Sample) : 7043 clients, 21 colonnes, sans aucune ligne dupliquée ni identifiant client dupliqué.

Profiling et détection de fuite

La colonne TotalCharges, bien que numérique par nature, est stockée en texte et présente 11 valeurs vides (0,16 % du dataset) — la vérification a montré que ces 11 clients ont tous une ancienneté (tenure) de 0 mois : ce sont des clients tout juste souscrits, jamais encore facturés. Il s'agit d'une valeur manquante logique, non d'une anomalie, traitée par imputation dans le pipeline. Aucune

feature n'est connue uniquement après le départ du client ; l'identifiant customerID, sans pouvoir prédictif, a été exclu dès le chargement des données. Aucune fuite n'a été détectée à ce stade.

Analyse bivariable

Le taux de churn varie fortement selon plusieurs variables. Les contrats mensuels (Month-to-month) affichent un taux de churn de 42,7 %, contre 11,3 % pour les contrats annuels et seulement 2,8 % pour les contrats biannuels. Le mode de paiement par chèque électronique est associé à un taux de churn de 45,3 %, contre 15 à 19 % pour les autres modes. L'absence de services de sécurité en ligne ou de support technique double quasiment le taux de churn (environ 42 % sans le service, contre 15 % avec).

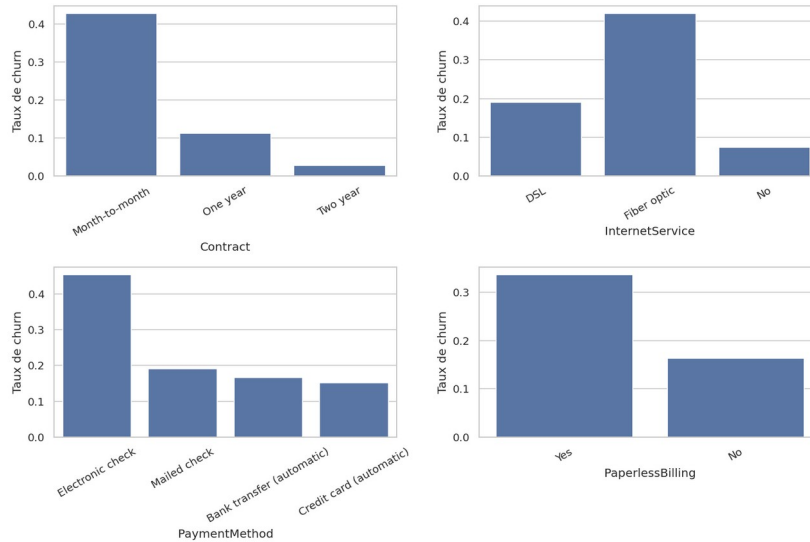


Figure 1 — Taux de churn par type de contrat, service Internet, mode de paiement et facturation dématérialisée.

Pouvoir discriminant (information mutuelle)

Le classement par information mutuelle — préféré à la corrélation de Pearson car il capture des dépendances non linéaires et s'applique aux variables catégorielles — place le type de contrat en tête, suivi de l'ancienneté, puis des services de sécurité et de support (OnlineSecurity, TechSupport, OnlineBackup). L'ancienneté et les charges se distinguent également nettement selon le statut de churn, comme le montre la figure 2 ci-dessous.

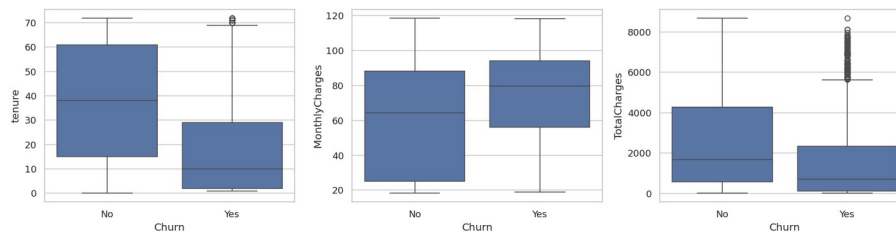


Figure 2 — Distribution de l'ancienneté, des charges mensuelles et des charges totales selon le statut de churn.

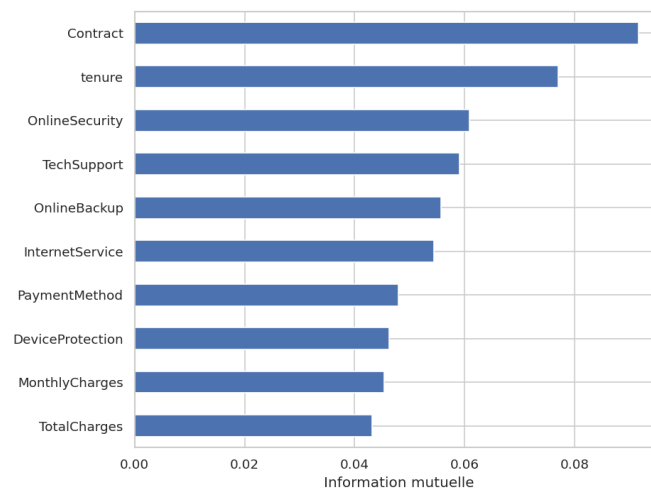


Figure 3 — Classement des dix features les plus discriminantes par information mutuelle.

Synthèse — trois insights majeurs

- Le type de contrat est le facteur dominant : l'absence d'engagement contractuel est le principal levier de risque de churn.
- L'ancienneté est le deuxième facteur le plus prédictif : les nouveaux clients partent nettement plus, ce qui suggère un enjeu d'onboarding dans les premiers mois.
- Les services de « verrou » (sécurité en ligne, support technique, sauvegarde) réduisent fortement le churn — un client équipé de plusieurs services est plus ancré et moins volatile.

Mission 2 — Préparation et pipeline sans fuite de données

Cette mission constitue le point le plus strictement noté du cahier des charges : garantir qu'aucune transformation apprise (imputation, encodage, mise à l'échelle, feature engineering statistique) n'est calculée sur autre chose que les données d'entraînement.

Split avant toute transformation

Les données sont séparées en un ensemble d'entraînement (80 %, 5634 clients) et un ensemble de test (20 %, 1409 clients) de façon stratifiée, avant toute imputation, tout encodage et toute mise à l'échelle. Cette séparation garantit que le jeu de test représente un futur intact, jamais vu par aucune étape de préparation.

Architecture du pipeline

L'ensemble des transformations est encapsulé dans un unique objet Pipeline scikit-learn, composé d'un ColumnTransformer avec deux sous-pipelines : un sous-pipeline numérique (imputation par la médiane puis standardisation) et un sous-pipeline catégoriel (imputation par le mode puis encodage one-hot). Toutes les statistiques sont apprises exclusivement sur les données d'entraînement, lors de l'appel fit(), puis réappliquées telles quelles sur les données de test.

Baseline et feature engineering

Une régression logistique entraînée dans ce pipeline obtient un F1-score de 0,633 ($\pm 0,024$) en validation croisée à 5 plis. Deux features ont été créées, NumServices (nombre de services de « verrou » souscrits) et ChargesPerTenure (ratio des charges mensuelles sur l'ancienneté), et leur apport a été mesuré honnêtement :

Configuration	F1 (CV, 5 plis)
Sans feature engineering	0,633 $\pm$ 0,024
Avec feature engineering	0,629 $\pm$ 0,020

Le gain mesuré est de -0,004, non significatif au vu de l'écart-type — conformément au principe du rasoir d'Occam explicitement demandé par le sujet, ces deux features n'apportent aucun bénéfice mesurable à un modèle linéaire simple. Elles ont néanmoins été conservées dans le pipeline pour être réévaluées avec des modèles non linéaires en Mission 3, plutôt que d'être artificiellement présentées comme un gain qu'elles ne produisent pas.

Mission 3 — Modélisation et comparaison rigoureuse

Baseline naïve et benchmark

Un classifieur naïf prédisant systématiquement la classe majoritaire obtient un rappel de 0,000 sur la classe churn — cette baseline est battue significativement par les trois modèles testés, entraînés dans le pipeline complet avec les mêmes plis de validation croisée stratifiée (StratifiedKFold, 5 plis).

Modèle	F1 moyen (CV)	Écart-type
Régression logistique	0,632	0,024
Forêt aléatoire	0,610	0,028
SVM (noyau RBF)	0,630	0,019

Justification du choix des trois familles : la régression logistique offre un point de référence interprétable, adapté à des relations globalement monotones (contrat, ancienneté) ; la forêt aléatoire capture des interactions non linéaires entre variables catégorielles nombreuses ; le SVM à noyau RBF explore une frontière de décision non linéaire différente, complémentaire à la forêt aléatoire pour la comparaison.

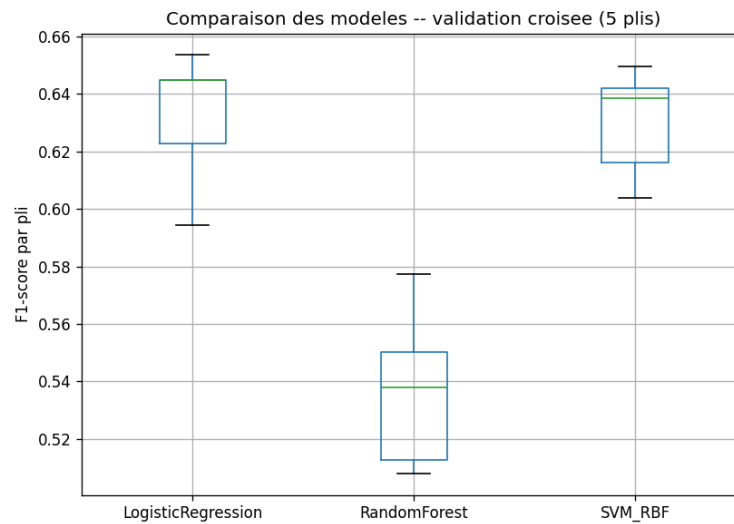


Figure 4 — Distribution du F1-score par pli de validation croisée pour les trois modèles comparés.

Test statistique et analyse d'erreurs

Un test de Wilcoxon appliqué aux scores par pli entre les deux meilleurs modèles (régression logistique et SVM) donne une p-valeur de 0,625, très supérieure au seuil de 0,05 : la différence n'est pas statistiquement significative — ils sont considérés comme équivalents à ce stade, avant tuning. Sur le jeu de test, 362 clients sur 1409 sont mal classés (25,7 %) ; 83,4 % d'entre eux sont en contrat mensuel (contre environ 55 % dans l'ensemble du jeu de test), avec une ancienneté moyenne plus faible (23,0 contre 31,9 mois pour l'ensemble).

Mission 4 — Optimisation, calibration et interprétabilité

Tuning des hyperparamètres (Optuna)

Un espace de recherche portant sur sept hyperparamètres de la régression logistique (C, penalty, class_weight, max_iter, tol, fit_intercept, intercept_scaling) a été exploré sur 60 essais, avec un échantillonneur TPE et un élagueur MedianPruner permettant d'arrêter prématurément les essais peu prometteurs sur la base des scores intermédiaires par pli. Le meilleur F1 obtenu est de 0,636, contre 0,632 avec les paramètres par défaut. L'analyse d'importance montre que class_weight domine très largement (environ 81 %) : sur un problème déséquilibré (26,5 % de churn), le rééquilibrage des classes pèse bien davantage que la régularisation fine.

Calibration

Le modèle non calibré est visiblement sur-confiant dans les tranches de probabilité moyennes-hautes. Après application d'une calibration isotonique (CalibratedClassifierCV, validation croisée à 5 plis), l'erreur de calibration moyenne passe de 0,164 à 0,024 — une amélioration nette, illustrée par le reliability diagram ci-dessous. Une probabilité de churn fiable est essentielle dans ce contexte métier : elle permet au service marketing de doser la générosité de l'offre de rétention selon le niveau de risque réel du client.

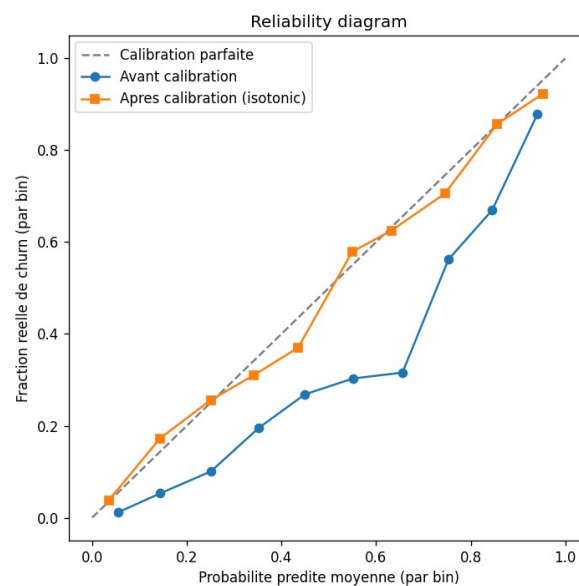


Figure 5 — Reliability diagram avant et après calibration isotonique.

Interprétabilité (SHAP)

L'analyse SHAP confirme et affine les observations de la Mission 1 : l'ancienneté (tenure) domine largement l'importance globale, suivie par le type de contrat (Month-to-month et Two year) et les charges mensuelles. Les valeurs faibles de tenure poussent fortement la prédiction vers le churn, tandis que les valeurs élevées la poussent vers la fidélité — une relation monotone conforme à l'intuition métier.

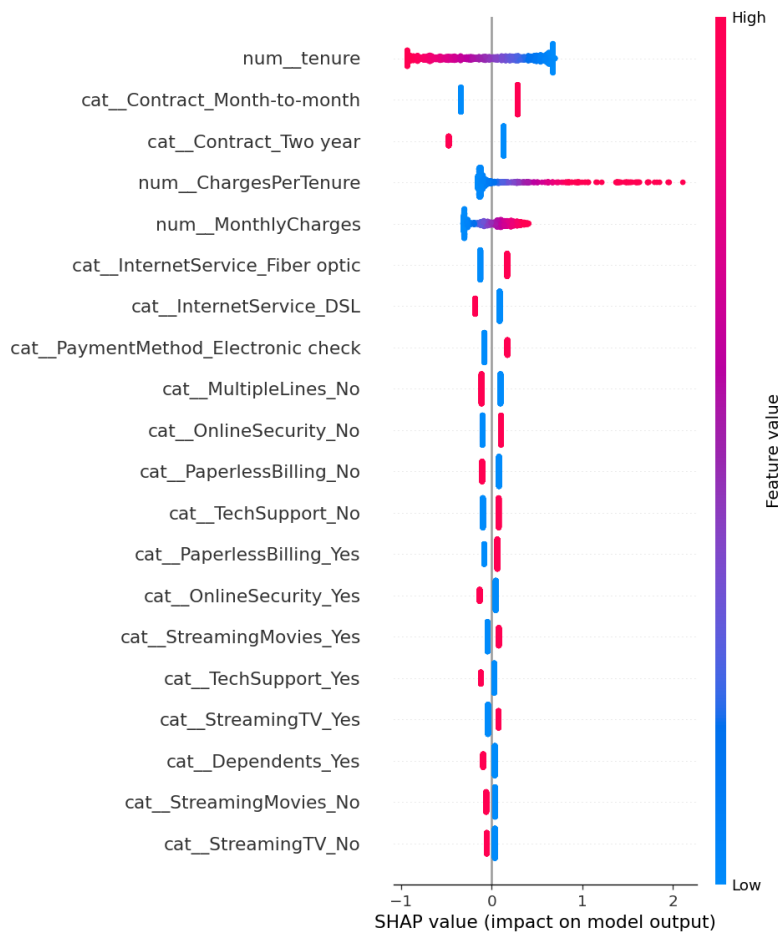


Figure 6 — Summary plot SHAP : impact de chaque feature sur la prédiction de churn.

Les explications individuelles (un vrai positif, un vrai négatif, un faux positif) et le dependence plot de la feature dominante ont également été produits — l'exemple du faux positif révèle un client d'ancienneté faible mais disposant d'un contrat engageant ou de plusieurs services souscrits, signaux contradictoires que le modèle pondère mal dans ce cas précis. Le détail complet de ces quatre visuels est disponible dans le dossier reports/ du dépôt Git.

Choix du seuil de décision

Au seuil par défaut de 0,5, le F2-score n'atteint que 0,566. En optimisant explicitement le seuil selon le F2-score — cohérent avec l'asymétrie de coûts fixée en Mission 0 — le seuil optimal retenu est de 0,119, pour une précision de 0,435 et un rappel de 0,925. Ce seuil, très inférieur à 0,5, traduit numériquement le déséquilibre des coûts établi dès la Mission 0 : il est préférable de sur-cibler la campagne de rétention que de laisser partir des clients à risque.

Mission 5 — Déploiement, tests et monitoring

Sérialisation et tests automatisés

Le pipeline final calibré et le seuil de décision retenu sont sérialisés avec joblib. Six tests pytest couvrent l'ensemble des exigences du cahier des charges : la forme de la sortie, l'appartenance des probabilités à l'intervalle $[0, 1]$, la robustesse aux valeurs manquantes, la présence des features attendues, une performance minimale sur un mini-jeu de référence, et le déterminisme du pipeline rechargé.

Preuve d'exécution — tests en local (Windows, Python 3.14)

Exécution de la suite complète depuis un terminal PowerShell, sur la machine de développement :

```
PS> pytest tests\ -v
===== test session starts =====
platform win32 -- Python 3.14.6, pytest-9.1.1
collected 6 items

tests/test_pipeline.py::test_output_shape PASSED
tests/test_pipeline.py::test_probabilities_in_range PASSED
tests/test_pipeline.py::test_handles_missing_values PASSED
tests/test_pipeline.py::test_expected_features_present PASSED
tests/test_pipeline.py::test_performance_on_reference_set PASSED
tests/test_pipeline.py::test_reloaded_pipeline_is_deterministic PASSED

===== 6 passed in 86.78s (0:01:26) =====
```

Preuve d'exécution — tests en intégration continue (GitHub Actions)

Un workflow GitHub Actions (.github/workflows/tests.yml) rejoue automatiquement l'ensemble du pipeline sur une machine Ubuntu propre (Python 3.12) à chaque push sur la branche main : installation des dépendances, téléchargement du dataset, réentraînement complet du modèle, puis exécution de pytest. La première exécution avait échoué (échec d'installation d'un paquet spécifique à Windows, pywinpty, présent par erreur dans un requirements.txt figé via pip freeze sous Windows) ; une fois requirements.txt corrigé pour rester portable entre systèmes d'exploitation, le workflow s'exécute avec succès et les 6 tests passent sur l'environnement Linux du serveur, indépendamment de la machine locale. Le badge de statut, visible en tête du README du dépôt, reflète en continu ce résultat, et l'historique complet des exécutions est consultable dans l'onglet Actions du dépôt GitHub.

- Étapes automatisées : récupération du code, installation de Python 3.12, installation des dépendances, téléchargement du dataset, réentraînement du pipeline, exécution de pytest.
- Déclenchement : à chaque push et à chaque pull request sur la branche main.
- Résultat observé : passage du statut Failure (échec de build pywinpty) à Success après correction, badge vert confirmé.

API REST (FastAPI)

Trois endpoints exposent le modèle en production : GET /health renvoie le statut du service et la version du modèle chargé ; POST /predict accepte les caractéristiques d'un client au format JSON et renvoie la probabilité de churn ainsi que la décision au seuil retenu ; GET /model-info expose les métadonnées du modèle. Les trois endpoints ont été testés avec succès, tant via des requêtes HTTP directes (Invoke-RestMethod, curl) qu'à travers la documentation interactive générée automatiquement par FastAPI (/docs).

Documentation et monitoring

Un README décrit le problème, les données, le modèle, les instructions d'installation et de lancement de l'API. Une model card documente les données d'entraînement, la performance de validation, les limites connues (absence d'audit d'équité formel, hypothèse de stationnarité, seuil basé sur une estimation de coûts) et l'usage prévu, conformément au standard introduit par Mitchell et al. (2019). Le plan de monitoring recommande la surveillance du data drift, du concept drift et du performance drift via l'outil open-source Evidently AI, avec un réentraînement déclenché si le F1 réel descend sous 0,55 sur deux mois consécutifs.

Questions de réflexion

1. Concept drift

Un modèle bon à l'entraînement peut se dégrader en production si la relation entre les features et la cible change dans le temps. Exemple : une offre concurrente ciblant les contrats biannuels pourrait faire cherner des clients jusque-là jugés sûrs, sans que le modèle le détecte. Ce qu'il faut surveiller est la performance réelle constatée a posteriori, pas seulement la stabilité des données d'entrée.

2. Information mutuelle et corrélation de Pearson

Pearson ne mesure que les relations linéaires entre variables numériques. L'information mutuelle capture toute forme de dépendance — linéaire, non linéaire, catégorielle — d'où son usage en Mission 1 pour classer l'ensemble des features.

3. No Free Lunch

Aucun algorithme n'est universellement meilleur sur tous les problèmes : la performance dépend de la structure des données. Ce théorème justifie le benchmark de la Mission 3, où la régression logistique égale ou dépasse la forêt aléatoire par défaut.

4. Équité (fairness)

Aucun audit d'équité formel n'a été mené — limite documentée dans la model card. La demographic parity (taux de prédiction positive par sous-groupe) et l'equalized odds (rappel et faux positifs par sous-groupe) permettraient de le faire, en filtrant le jeu de test par sous-groupe.

5. Fuite de données — vérification

- Imputation, scaling, encodage : calculés uniquement sur le train, via un Pipeline unique fit après le split.
- Feature engineering : intégré au même Pipeline, appliqué identiquement sur train et test.
- Validation croisée du tuning : le Pipeline complet est réentraîné à chaque pli.
- customerID : exclu dès le chargement des données.
- Calibration : CalibratedClassifierCV utilise son propre mode de validation croisée interne, séparé de l'entraînement du modèle.

Conclusion

Ce projet a permis de construire, de bout en bout, un système complet de prédiction du churn client, en respectant l'enchaînement méthodologique imposé : cadrage métier préalable, exploration rigoureuse, pipeline sans fuite, comparaison honnête de modèles, optimisation et interprétation, puis déploiement testé et documenté.

Les résultats confirment les hypothèses métier de la Mission 0 : le type de contrat et l'ancienneté dominant le pouvoir prédictif, confirmés par l'EDA et par l'interprétation SHAP. Le seuil de décision optimisé (0,119, très inférieur à 0,5) traduit numériquement l'asymétrie des coûts d'erreur identifiée dès le cadrage.

Au-delà du score, ce projet a été l'occasion de mettre en pratique une discipline d'ingénieur data : ne jamais faire confiance à un gain de performance sans vérifier l'absence de fuite, documenter honnêtement les résultats négatifs (le feature engineering de la Mission 2 n'a pas amélioré le score, et cela a été assumé), et livrer un système utilisable, pas seulement un notebook expérimental. Les limites identifiées — absence d'audit d'équité, coûts d'erreur estimés — constituent des axes d'amélioration pour une itération future.

Bibliographie et ressources

- Géron, A. Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow. O'Reilly Media.
- Mitchell, M. et al. (2019). « Model Cards for Model Reporting ». FAccT.
- Breck, E. et al. (2017). « The ML Test Score ». Google.
- Documentation officielle : scikit-learn.org, optuna.org, shap.readthedocs.io, fastapi.tiangolo.com, evidentlyai.com.
- IBM Sample Data Sets — Telco Customer Churn (identique au dataset Kaggle blastchar/telco-customer-churn).
- SY, I. (2025–2026). Supervised Learning — Cours pratique. ISI, Master Intelligence Artificielle.

Annexe A — Dépôt Git et reproduction

Le code source complet, les notebooks, les tests, l'API et l'historique des exécutions (intégration continue) sont disponibles dans le dépôt Git du projet — voir le lien communiqué avec ce rapport et le README à sa racine.

Structure du dépôt

```
De la Donnée à la Décision/
├── .github/workflows/  tests.yml – integration continue (CI)
├── data/               CSV brut (telecharge, non versionne)
├── notebooks/         01_eda.py – exploration (Mission 1)
├── src/               pipeline.py, download_data.py,
│                     train_baseline.py, train_models.py,
│                     tune_and_explain.py
├── tests/            test_pipeline.py – 6 tests pytest
├── api/              main.py – API FastAPI (3 endpoints)
├── model/            pipeline_final.joblib, MODEL_CARD.md
├── reports/          livrables detaillés, mission par mission
├── requirements.txt
└── README.md
```

Commandes pour tout reproduire, de A à Z

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python src\download_data.py      # téléchargement du dataset
python src\train_baseline.py     # pipeline + baseline (M2)
python src\train_models.py       # benchmark 3 modèles (M3)
python src\tune_and_explain.py   # tuning, calibration, SHAP (M4)
pytest tests\ -v                 # suite de tests (M5)
uvicorn api.main:app --reload    # lancement de l'API (M5)
```